# Systèmes d'exploitation

Axel Guérin

# Chapitre 1 — Comprendre l'ordinateur et le système d'exploitation

1. Qu'est-ce qu'un système informatique ?
2. Les principaux composants matériels
3. Qu'est-ce qu'un système d'exploitation ?
4. Les rôles du système d'exploitation
5. Le noyau (kernel)
6. Applications, programmes et processus
7. Les principaux systèmes d'exploitation

1. **Qu'est-ce qu'un système informatique ?**
2. Les principaux composants matériels
3. Qu'est-ce qu'un système d'exploitation ?
4. Les rôles du système d'exploitation
5. Le noyau (kernel)
6. Applications, programmes et processus
7. Les principaux systèmes d'exploitation

# Qu'est-ce qu'un système informatique ?

## Définition

Un **système informatique** est un ensemble de composants **matériels** et **logiciels** permettant de traiter, stocker et échanger de l'information.

On distingue donc deux grandes composantes :

## Matériel (Hardware)

Les composants physiques :

- processeur ;
- mémoire ;
- stockage ;
- périphériques.

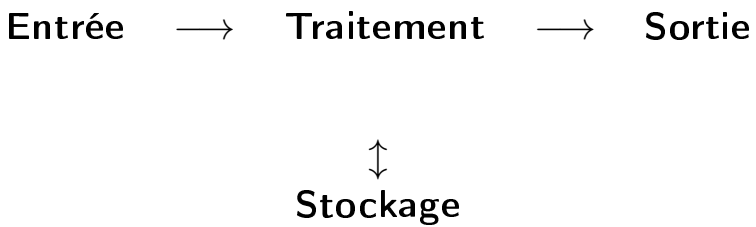
## Logiciel (Software)

Les programmes :

- système d'exploitation ;
- applications ;
- outils et utilitaires.

# Que fait un système informatique ?

Un système informatique réalise principalement quatre grandes fonctions :




- **Entrée :** clavier, souris, réseau, capteurs...
- **Traitement :** calculs et exécution des programmes.
- **Stockage :** mémoire, SSD, disque dur...
- **Sortie :** écran, son, fichiers, réseau...

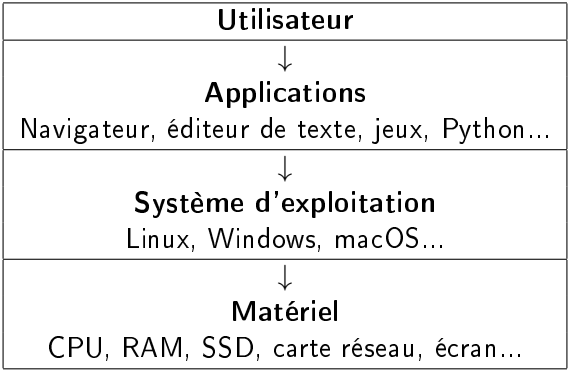
# Exemple : regarder une vidéo

Que se passe-t-il lorsque vous regardez une vidéo sur votre ordinateur ?

1. Les données arrivent depuis Internet.
2. Elles sont temporairement stockées en mémoire.
3. Le processeur exécute les programmes nécessaires.
4. Les données vidéo et audio sont décodées.
5. L'image est envoyée vers l'écran.
6. Le son est envoyé vers le périphérique audio.

Toutes ces opérations nécessitent la coopération de nombreux composants matériels et logiciels.

# Les différentes couches d'un système informatique




Le **système d'exploitation** joue un rôle central : il fait le lien entre les applications et le matériel.

# Un système informatique n'est pas forcément un PC

On retrouve des systèmes informatiques dans de très nombreux appareils :

- ordinateurs de bureau et ordinateurs portables ;
- smartphones et tablettes ;
- serveurs ;
- consoles de jeux ;
- télévisions connectées ;
- voitures ;
- objets connectés ;
- supercalculateurs.

## Point commun

Malgré leurs différences, ces systèmes possèdent généralement du **matériel**, exécutent des **programmes** et doivent gérer leurs différentes **ressources**.

# Qui coordonne tout cela ?

Un ordinateur peut avoir simultanément :

- plusieurs programmes en cours d'exécution ;
- plusieurs milliards d'octets en mémoire ;
- des milliers de fichiers ;
- de nombreux périphériques ;
- plusieurs utilisateurs ;
- plusieurs connexions réseau.

**Problème**

Qui décide quel programme utilise le processeur ?
Qui gère la mémoire, les fichiers et les périphériques ?

$\Longrightarrow$ **Le système d'exploitation**

1. Qu'est-ce qu'un système informatique ?
2. **Les principaux composants matériels**
3. Qu'est-ce qu'un système d'exploitation ?
4. Les rôles du système d'exploitation
5. Le noyau (kernel)
6. Applications, programmes et processus
7. Les principaux systèmes d'exploitation

# Les principaux composants matériels

Un ordinateur est constitué de plusieurs composants spécialisés qui communiquent entre eux.

- **Processeur (CPU)** : exécute les instructions.
- **Mémoire vive (RAM)** : conserve temporairement les données utilisées.
- **Stockage** : conserve durablement les données.
- **Carte mère** : permet aux composants de communiquer.
- **Processeur graphique (GPU)** : réalise des calculs massivement parallèles.
- **Périphériques** : permettent les interactions avec l'extérieur.

Chaque composant possède un rôle différent, mais leur fonctionnement doit être **coordonné**.

# Le processeur (CPU)

## CPU — Central Processing Unit

Le processeur est le composant chargé d'**exécuter les instructions** des programmes.

Il réalise notamment :

- des opérations arithmétiques ;
- des opérations logiques ;
- des comparaisons ;
- des accès à la mémoire ;
- le contrôle de l'exécution des programmes.

Un processeur moderne possède généralement plusieurs **cœurs** capables d'exécuter des tâches en parallèle.

# La mémoire vive (RAM)

## RAM — Random Access Memory

La RAM contient les **programmes et données actuellement utilisés** par l'ordinateur.

Elle est :

- **rapide** ;
- directement accessible par le processeur ;
- de capacité limitée ;
- **volatile**.

## Mémoire volatile

Le contenu de la RAM disparaît lorsque l'ordinateur est éteint.

Lorsqu'un programme est lancé, une partie de son contenu et de ses données est chargée depuis le stockage vers la RAM.

# Le stockage

## Stockage

Le stockage permet de conserver les **programmes et les données de manière durable**.

Principales technologies :

- **SSD** : mémoire flash, rapide et sans pièce mécanique ;
- **HDD** : disque magnétique mécanique ;
- clés USB et cartes mémoire ;
- supports externes.

Contrairement à la RAM, les données restent présentes lorsque l'ordinateur est éteint.

## Exemples de données stockées

Système d'exploitation, logiciels, documents, photos, vidéos, code source...

# RAM et stockage : quelle différence ?

| | RAM | SSD / HDD |
|---|---|---|
| Vitesse | Très rapide | Plus lent |
| Capacité | Plus faible | Plus importante |
| Volatile | Oui | Non |
| Utilisation | Données en cours | Données persistantes |

Lorsqu'un programme est lancé :

**Stockage** $\longrightarrow$ **RAM** $\longrightarrow$ **CPU**

Le programme est **stocké** sur le disque, **chargé** en mémoire, puis ses instructions sont **exécutées** par le processeur.

# La carte mère

## Carte mère

La carte mère est le circuit principal sur lequel sont connectés les différents composants de l'ordinateur.

Elle permet notamment de connecter :

- le processeur ;
- la mémoire RAM ;
- les supports de stockage ;
- les cartes d'extension ;
- les périphériques.

Les composants communiquent grâce à différents **bus et interfaces de communication**.

# Le processeur graphique (GPU)

## GPU — Graphics Processing Unit

Le GPU est un processeur spécialisé capable d'effectuer un grand nombre de calculs en parallèle.

Historiquement, son rôle principal est le calcul nécessaire à l'affichage des images.

Aujourd'hui, les GPU sont également utilisés pour :

- les jeux vidéo et la 3D ;
- le traitement d'images et de vidéos ;
- les simulations scientifiques ;
- l'intelligence artificielle.

## CPU vs GPU

Le CPU possède quelques cœurs très polyvalents tandis qu'un GPU dispose d'un très grand nombre d'unités de calcul adaptées aux traitements parallèles.

# Les périphériques

Les périphériques permettent à l'ordinateur de **communiquer avec son environnement**.

## Périphériques d'entrée

- clavier ;
- souris ;
- microphone ;
- webcam ;
- capteurs.

## Périphériques de sortie

- écran ;
- haut-parleurs ;
- imprimante ;
- vidéoprojecteur.

Certains périphériques permettent à la fois l'entrée et la sortie d'informations : écran tactile, carte réseau, support USB...

# Comment gérer tous ces composants ?

Nous disposons maintenant de nombreuses ressources :

- plusieurs cœurs de processeur ;
- plusieurs gigaoctets de mémoire ;
- des supports de stockage ;
- un processeur graphique ;
- de nombreux périphériques.

Mais plusieurs programmes peuvent vouloir utiliser **les mêmes ressources simultanément**.

**Quelques questions**

Qui décide quel programme utilise le processeur ?
Qui répartit la mémoire ?
Qui permet à un programme d'accéder au disque, au réseau ou à l'écran ?

$\Longrightarrow$ **Le système d'exploitation**

1. Qu'est-ce qu'un système informatique ?
2. Les principaux composants matériels
3. **Qu'est-ce qu'un système d'exploitation ?**
4. Les rôles du système d'exploitation
5. Le noyau (kernel)
6. Applications, programmes et processus
7. Les principaux systèmes d'exploitation

# Qu'est-ce qu'un système d'exploitation ?

**Définition**

Un **système d'exploitation** (Operating System ou **OS**) est un ensemble de programmes qui permet de gérer les ressources matérielles d'un ordinateur et de fournir des services aux applications.

Il constitue une interface entre :

**Applications** $\longleftrightarrow$ **Système d'exploitation** $\longleftrightarrow$ **Matériel**

**Exemples**

Windows, GNU/Linux, macOS, Android et iOS sont des systèmes d'exploitation.

# Pourquoi avons-nous besoin d'un système d'exploitation ?

Imaginons un ordinateur **sans système d'exploitation**.

Chaque programme devrait savoir directement :

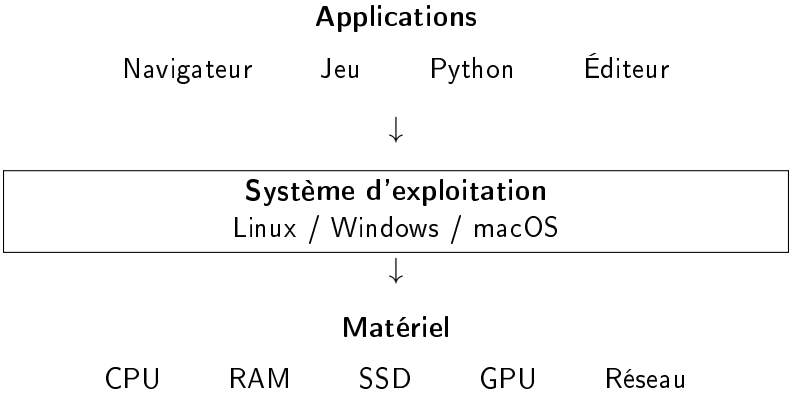
- communiquer avec le processeur ;
- utiliser la mémoire ;
- lire et écrire sur le disque ;
- communiquer avec le clavier et la souris ;
- afficher une image à l'écran ;
- utiliser la carte réseau ;
- gérer les autres programmes.

**Problème**

Chaque application devrait connaître le fonctionnement précis du matériel de l'ordinateur.

# Le système d'exploitation comme intermédiaire

Le système d'exploitation fournit une **interface commune** permettant aux programmes d'utiliser le matériel.



# Exemple : enregistrer un fichier

Vous écrivez un programme Python :

```
fichier.write("Bonjour")
```

Votre programme ne contrôle pas directement le SSD.

**Programme Python**

↓

demande d'écriture

↓

**Système d'exploitation**

↓

gestion du système de fichiers et du périphérique

↓

**SSD**

# Plusieurs programmes, une seule machine

Sur un ordinateur, de nombreux programmes peuvent fonctionner en même temps :

- un navigateur Web ;
- un lecteur de musique ;
- un éditeur de code ;
- un terminal ;
- des services fonctionnant en arrière-plan.

Pourtant, ils utilisent tous les mêmes ressources :

**CPU** **RAM** **Stockage** **Réseau** **Périphériques**

Le système d'exploitation doit permettre le **partage** et la **coordination** de ces ressources.

# Le système d'exploitation au centre de la machine

Nous pouvons donc retenir :

## Système d'exploitation

Le système d'exploitation :

- se situe entre les applications et le matériel ;
- fournit des services aux programmes ;
- masque une partie de la complexité du matériel ;
- coordonne l'utilisation des ressources.

Mais concrètement, que doit-il gérer ?

Processeur ? Mémoire ? Fichiers ?

Périphériques ? Utilisateurs ?

1. Qu'est-ce qu'un système informatique ?
2. Les principaux composants matériels
3. Qu'est-ce qu'un système d'exploitation ?
4. **Les rôles du système d'exploitation**
5. Le noyau (kernel)
6. Applications, programmes et processus
7. Les principaux systèmes d'exploitation

# Les rôles du système d'exploitation

Le système d'exploitation doit gérer et partager les différentes **ressources** de l'ordinateur.

Ses principaux rôles sont :

- gérer l'exécution des **programmes et processus** ;
- gérer le **processeur** ;
- gérer la **mémoire** ;
- organiser les **fichiers et le stockage** ;
- gérer les **périphériques** ;
- gérer les **utilisateurs et les droits d'accès** ;
- fournir des services de **communication et réseau**.

Le système d'exploitation joue le rôle de **chef d'orchestre** entre les programmes et les ressources matérielles.

# Gérer l'exécution des programmes

Lorsque nous lançons un programme, le système d'exploitation doit organiser son exécution.

## Processus

Un **processus** est une instance d'un programme en cours d'exécution.

Le système d'exploitation doit notamment :

- créer les processus ;
- suivre leur état ;
- leur attribuer des ressources ;
- permettre leur communication ;
- terminer les processus lorsqu'ils s'arrêtent.

## Exemple

Ouvrir trois fenêtres d'un navigateur peut entraîner l'exécution de plusieurs processus.

# Partager le processeur

De nombreux processus peuvent vouloir utiliser le processeur **en même temps**.

Le système d'exploitation décide :

- quel processus peut utiliser le CPU ;
- pendant combien de temps ;
- dans quel ordre les processus sont exécutés.

Processus A $\rightarrow$ Processus B $\rightarrow$ Processus C $\rightarrow$ Processus A $\rightarrow$ $\cdots$

## Ordonnancement

Le mécanisme permettant de choisir les processus à exécuter est appelé **ordonnancement** (*scheduling*).

# Comment plusieurs programmes fonctionnent-ils simultanément ?

Vous pouvez simultanément :

- écouter de la musique ;
- naviguer sur Internet ;
- télécharger un fichier ;
- écrire du code ;
- exécuter un programme.

Le système d'exploitation alterne rapidement entre les différents processus et exploite les différents cœurs du processeur.

**Multitâche**

Un système **multitâche** permet l'exécution concurrente de plusieurs programmes.

Pour l'utilisateur, ces programmes donnent l'impression de fonctionner simultanément.

# Gérer la mémoire

Chaque programme en cours d'exécution a besoin de mémoire.

Le système d'exploitation doit :

- attribuer de la mémoire aux processus ;
- suivre les zones utilisées et disponibles ;
- récupérer la mémoire lorsqu'un processus se termine ;
- empêcher un processus d'accéder librement à la mémoire d'un autre processus.

| RAM |
| --- |
| Système d'exploitation |
| Navigateur |
| Éditeur de code |
| Programme Python |
| Mémoire disponible |

# Et lorsque la mémoire vient à manquer ?

La quantité de RAM disponible est limitée.

Le système d'exploitation utilise différentes techniques pour gérer efficacement cette mémoire.

## Mémoire virtuelle

La **mémoire virtuelle** permet notamment de fournir à chaque processus son propre espace d'adressage et d'utiliser, si nécessaire, une partie du stockage comme extension de la mémoire.

**Processus** $\longrightarrow$ **Mémoire virtuelle** $\longrightarrow$ **RAM / Stockage**

Le stockage est beaucoup plus lent que la RAM : il ne remplace donc pas réellement la mémoire vive.

# Gérer les fichiers et le stockage

Le système d'exploitation doit permettre d'organiser et de retrouver les données stockées.

Il fournit notamment :

- des **fichiers** ;
- des **répertoires** ;
- une organisation hiérarchique ;
- des noms et des chemins ;
- des droits d'accès ;
- des opérations de lecture et d'écriture.

**Exemple**

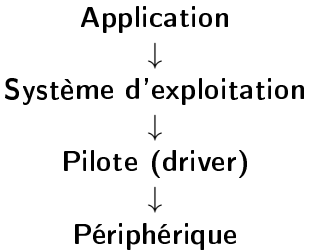
Lorsque vous ouvrez
`/home/etudiant/Documents/cours.pdf`
le système d'exploitation doit retrouver les données correspondantes sur le support de stockage.

# Gérer les périphériques

Les périphériques peuvent être très différents :

Clavier Souris SSD Écran Imprimante Carte réseau

Le système d'exploitation fournit une interface permettant aux programmes de les utiliser.



# Les pilotes de périphériques

## Pilote (driver)

Un **pilote** est un logiciel permettant au système d'exploitation de communiquer avec un périphérique matériel.

Le pilote connaît les particularités du matériel.

Le système d'exploitation peut ainsi fournir aux applications une interface plus uniforme.

## Exemple

Un logiciel n'a pas besoin de connaître le fonctionnement électronique précis de chaque modèle d'imprimante ou de carte réseau.

# Gérer les utilisateurs

Un système d'exploitation peut être utilisé par plusieurs **utilisateurs**.

Chaque utilisateur peut disposer :

- d'un identifiant ;
- d'un espace personnel ;
- de ses propres fichiers ;
- de ses préférences ;
- de différents droits d'accès.

## Sous Linux

Chaque utilisateur possède généralement un répertoire personnel :

```
/home/alice
/home/bob
```

# Contrôler les accès

Tous les utilisateurs et programmes ne doivent pas pouvoir effectuer toutes les opérations.

Le système d'exploitation contrôle notamment :

- qui peut lire un fichier ;
- qui peut le modifier ;
- qui peut exécuter un programme ;
- qui peut accéder à certaines ressources ;
- qui peut modifier la configuration du système.

**Principe**

Les ressources du système sont associées à des **permissions**.

Nous retrouverons cette notion avec les permissions Linux :

```
rwx
```

# Gérer les communications réseau

Le système d'exploitation fournit également les mécanismes permettant aux programmes de communiquer sur un réseau.

Il gère notamment :

- les interfaces réseau ;
- les connexions ;
- les protocoles réseau ;
- l'envoi et la réception des données.

**Navigateur** $\longrightarrow$ **OS** $\longrightarrow$ **Carte réseau** $\longrightarrow$ **Internet**

Les applications utilisent ainsi les services réseau fournis par le système d'exploitation.

# Les rôles du système d'exploitation — Synthèse

| Ressource | Rôle du système |
|---|---|
| CPU | Ordonnancer les processus |
| RAM | Attribuer et protéger la mémoire |
| Stockage | Organiser les fichiers |
| Périphériques | Contrôler les accès au matériel |
| Utilisateurs | Gérer les comptes et permissions |
| Réseau | Permettre les communications |

**À retenir**

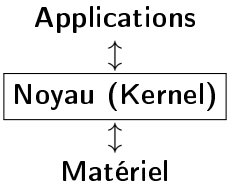
Le système d'exploitation **abstrait, partage, protège et coordonne les ressources** de l'ordinateur.

1. Qu'est-ce qu'un système informatique ?
2. Les principaux composants matériels
3. Qu'est-ce qu'un système d'exploitation ?
4. Les rôles du système d'exploitation
5. **Le noyau (kernel)**
6. Applications, programmes et processus
7. Les principaux systèmes d'exploitation

# Le noyau : le cœur du système d'exploitation

## Définition

Le **noyau** (*kernel*) est la partie centrale du système d'exploitation.
Il gère directement les principales ressources matérielles et fournit des services aux programmes.




Le noyau est chargé en mémoire au démarrage de l'ordinateur et reste actif pendant toute son utilisation.

# Système d'exploitation et noyau

Le **noyau** et le **système d'exploitation** ne désignent pas exactement la même chose.

| **Applications** |
| --- |
| Navigateur, éditeur, terminal... |
| **Outils et bibliothèques système** |
| Shell, commandes, services... |
| **Noyau** |
| Processus, mémoire, fichiers, périphériques... |
| **Matériel** |
| CPU, RAM, stockage, périphériques... |

## À retenir

Le noyau est une **partie fondamentale** du système d'exploitation, mais le système d'exploitation comprend également de nombreux programmes et outils autour du noyau.

# Pourquoi les programmes passent-ils par le noyau ?

Une application ne doit pas pouvoir contrôler librement les ressources de l'ordinateur.

Imaginez qu'un programme puisse directement :

- modifier n'importe quelle zone de la mémoire ;
- lire les données d'un autre programme ;
- accéder librement au disque ;
- contrôler directement tous les périphériques ;
- arrêter d'autres programmes.

Cela poserait d'importants problèmes de **stabilité** et de **sécurité**.

Le noyau contrôle donc l'accès aux ressources sensibles.

# Mode utilisateur et mode noyau

Les processeurs modernes proposent différents niveaux de privilèges.

## Mode utilisateur

Les applications s'exécutent avec des **droits limités**.

- Navigateur
- Éditeur
- Programme Python
- Shell

## Mode noyau

Le noyau possède des **privilèges élevés**.

- accès au matériel ;
- gestion de la mémoire ;
- gestion du CPU ;
- gestion des périphériques.

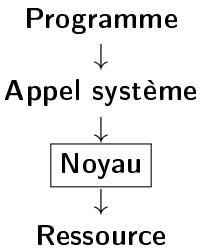
Cette séparation permet notamment d'éviter qu'une application puisse compromettre directement l'ensemble du système.

# Comment demander un service au noyau ?

Une application doit régulièrement demander au noyau d'effectuer certaines opérations.

## Appel système

Un **appel système** (*system call*) est un mécanisme permettant à un programme de demander un service au noyau.



# Exemple : lire un fichier

Supposons qu'un programme souhaite lire un fichier.

1. Le programme demande l'ouverture du fichier.
2. Cette demande est transmise au noyau.
3. Le noyau vérifie les permissions.
4. Il localise les données sur le stockage.
5. Il récupère les données nécessaires.
6. Les données sont transmises au programme.

Programme $\rightarrow$ Noyau $\rightarrow$ Système de fichiers $\rightarrow$ Stockage

Même une opération aussi simple que lire un fichier implique donc l'intervention du système d'exploitation.

# Et Linux dans tout cela ?

## Linux

Au sens strict, **Linux est un noyau**, créé initialement par Linus Torvalds en 1991.

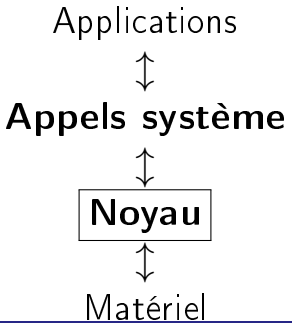
Autour du noyau Linux, on trouve de nombreux logiciels :

- des bibliothèques ;
- des outils système ;
- des shells ;
- des gestionnaires de paquets ;
- des interfaces graphiques ;
- des applications.

L'ensemble constitue ce que l'on appelle généralement une **distribution Linux**.

# Le noyau — À retenir

- Le noyau est le **cœur du système d'exploitation**.
- Il possède un accès privilégié aux ressources matérielles.
- Les applications s'exécutent généralement en **mode utilisateur**.
- Les programmes demandent des services au noyau grâce aux **appels système**.
- Le noyau contrôle et protège l'accès aux ressources.



1. Qu'est-ce qu'un système informatique ?
2. Les principaux composants matériels
3. Qu'est-ce qu'un système d'exploitation ?
4. Les rôles du système d'exploitation
5. Le noyau (kernel)
6. **Applications, programmes et processus**
7. Les principaux systèmes d'exploitation

# Applications, programmes et processus

Trois termes sont souvent utilisés pour parler des logiciels :

## Programme

Un **programme** est un ensemble d'instructions permettant de réaliser une tâche.
Il est généralement stocké dans un fichier sur le disque.

## Application

Une **application** est un logiciel destiné à fournir une ou plusieurs fonctionnalités à l'utilisateur.

## Processus

Un **processus** est une instance d'un programme **en cours d'exécution**.

# Programme et processus : quelle différence ?

Un programme stocké sur le disque est essentiellement un ensemble d'instructions et de données.

**Programme** $\xrightarrow{\text{exécution}}$ **Processus**

## Programme

- stocké sur le disque ;
- contenu relativement statique ;
- contient des instructions ;
- n'est pas nécessairement exécuté.

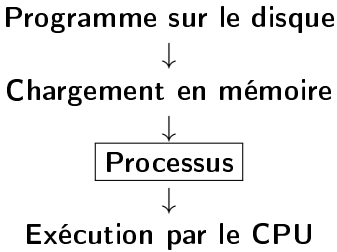
## Processus

- existe pendant l'exécution ;
- utilise de la mémoire ;
- utilise du temps processeur ;
- possède un état.

# Que se passe-t-il lorsque l'on lance un programme ?

Lorsque l'utilisateur lance un programme :

1. le système localise le programme sur le stockage ;
2. les éléments nécessaires sont chargés en mémoire ;
3. le système crée un nouveau processus ;
4. des ressources lui sont attribuées ;
5. le processeur commence à exécuter ses instructions.



# Un processus possède ses propres ressources

Un processus ne correspond pas uniquement aux instructions du programme.

Il possède notamment :

- du **code** à exécuter ;
- des **données** ;
- un espace en **mémoire** ;
- un état d'exécution ;
- des fichiers éventuellement ouverts ;
- des informations permettant au système de l'identifier et de le gérer.

Le système d'exploitation conserve donc des informations sur chaque processus actuellement présent dans le système.

# Identifier un processus : le PID

Le système d'exploitation doit pouvoir distinguer les différents processus.

**PID — Process Identifier**

Chaque processus possède un identifiant numérique appelé **PID**.

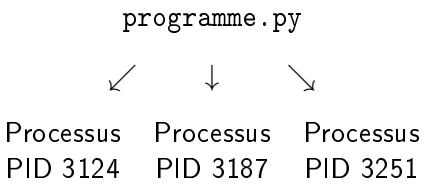
| Processus | PID |
|---|---|
| Navigateur | 2481 |
| Terminal | 3152 |
| Python | 4217 |
| Éditeur | 4582 |

Le PID permet au système et à l'utilisateur de **désigner précisément un processus**.

# Un programme peut donner plusieurs processus

Un même programme peut être exécuté plusieurs fois.

Par exemple, supposons que nous lançons trois fois le même programme Python :



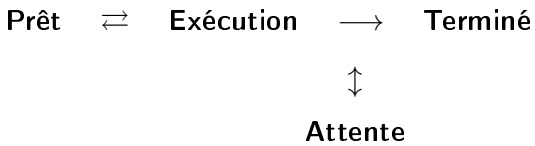

Les processus exécutent le même programme mais sont **des instances différentes** disposant de leurs propres ressources et de leur propre état.

# Les états d'un processus

Un processus n'utilise pas nécessairement le processeur en permanence.

Il peut se trouver dans différents états :

- **prêt** : il attend de pouvoir utiliser le processeur ;
- **en cours d'exécution** : ses instructions sont exécutées ;
- **en attente** : il attend un événement ou une ressource ;
- **terminé** : son exécution est terminée.



# Un processus ne calcule pas en permanence

Un programme peut devoir attendre :

- une saisie au clavier ;
- des données provenant du réseau ;
- la lecture d'un fichier ;
- une réponse d'un périphérique ;
- la fin d'une autre opération.

**Exemple**

Un navigateur qui attend la réponse d'un serveur Web n'a pas besoin d'utiliser continuellement le processeur.

Pendant cette attente, le système peut attribuer le processeur à **un autre processus**.

C'est l'un des principes permettant le fonctionnement **multitâche** du système.

# Observer et contrôler les processus

Le système d'exploitation permet à l'utilisateur d'observer et de contrôler certains processus.

Il est notamment possible de :

- afficher les processus en cours ;
- connaître leur PID ;
- observer leur consommation CPU et mémoire ;
- lancer un programme en arrière-plan ;
- demander l'arrêt d'un processus.

## Sous Linux

Nous utiliserons plus tard des commandes comme :

`ps` `top` `kill` `jobs`

# Programmes et processus — À retenir

**Programme** $\xrightarrow{\text{lancement}}$ **Processus**

- Un **programme** contient des instructions.
- Un **processus** est une instance d'un programme en cours d'exécution.
- Un même programme peut produire plusieurs processus.
- Chaque processus possède notamment un **PID**.
- Un processus peut être en exécution, prêt ou en attente.
- Le système d'exploitation organise leur exécution et leur attribue des ressources.

Le programme est le **code** ; le processus est **l'exécution de ce code**.

1. Qu'est-ce qu'un système informatique ?
2. Les principaux composants matériels
3. Qu'est-ce qu'un système d'exploitation ?
4. Les rôles du système d'exploitation
5. Le noyau (kernel)
6. Applications, programmes et processus
7. **Les principaux systèmes d'exploitation**

# Les principaux systèmes d'exploitation

Il existe de nombreux systèmes d'exploitation adaptés à différents usages.

Sur les ordinateurs personnels, trois grandes familles dominent :

- **Microsoft Windows** ;
- **macOS** ;
- **GNU/Linux**.

Mais on trouve également des systèmes d'exploitation dans :

- les smartphones ;
- les serveurs ;
- les supercalculateurs ;
- les systèmes embarqués ;
- les objets connectés.

## Windows

Windows est une famille de systèmes d'exploitation développée par **Microsoft**.

Il est principalement utilisé sur les ordinateurs personnels.

- Très répandu sur les PC.
- Système propriétaire.
- Large compatibilité matérielle et logicielle.
- Interface graphique fortement intégrée.
- Utilisé dans les environnements personnels et professionnels.

Windows propose également plusieurs environnements en ligne de commande :

`cmd.exe` `PowerShell`

# macOS

## macOS

macOS est le système d'exploitation développé par **Apple** pour ses ordinateurs Mac.

- Système propriétaire.
- Conçu spécifiquement pour le matériel Apple.
- Interface graphique fortement intégrée.
- Basé sur des technologies issues du monde Unix.

Cette origine Unix est particulièrement intéressante pour notre cours.

Le terminal macOS fournit un environnement proche de celui rencontré sous Linux.

## Shell par défaut

Les versions modernes de macOS utilisent généralement **Zsh** comme shell par défaut.

## Linux

Linux est un **noyau libre et open source**, créé initialement par Linus Torvalds en 1991.

Il est généralement utilisé avec de nombreux autres logiciels et outils pour former un système complet.

- libre et open source ;
- fortement utilisé sur les serveurs ;
- très présent dans le développement logiciel ;
- utilisé dans le calcul scientifique ;
- très présent dans les infrastructures Cloud ;
- disponible sur de nombreuses architectures matérielles.

# Les distributions Linux

Le noyau Linux est associé à différents logiciels, outils et systèmes de gestion pour former une **distribution Linux**.

## Quelques distributions

- Ubuntu
- Debian
- Fedora
- Linux Mint
- Arch Linux

# Et sur les smartphones ?

Les smartphones utilisent également des systèmes d'exploitation complets.

## Android

- développé principalement par Google ;
- utilise le noyau Linux ;
- utilisé par de nombreux constructeurs.

## iOS

- développé par Apple ;
- utilisé sur les iPhone ;
- partage des fondations avec macOS.

Un smartphone possède donc lui aussi un noyau, des processus, de la mémoire, un système de fichiers et des périphériques à gérer.

# Windows, macOS et Linux

| | **Windows** | **macOS** | **GNU/Linux** |
|---|---|---|---|
| Éditeur principal | Microsoft | Apple | Communauté / entreprises |
| Licence | Propriétaire | Propriétaire | Majoritairement libre |
| Matériel | PC | Mac | Très varié |
| Shell courant | PowerShell | Zsh | Bash / autres |
| Famille Unix | Non | Oui | Unix-like |

## Point important

Malgré leurs différences, ces systèmes remplissent fondamentalement les mêmes fonctions : **gérer les ressources de l'ordinateur et fournir des services aux applications**.

# Pourquoi allons-nous principalement utiliser Linux ?

Linux occupe une place importante dans de nombreux domaines de l'informatique :

- développement logiciel ;
- serveurs et administration système ;
- Cloud et infrastructures ;
- calcul scientifique ;
- intelligence artificielle et calcul haute performance ;
- systèmes embarqués.

Linux offre également un environnement particulièrement adapté à l'apprentissage de la **ligne de commande** et de l'**automatisation**.

## Objectif du cours

Nous utiliserons principalement Linux pour apprendre des concepts et des outils qui restent largement **transposables à macOS et Windows**.

# Chapitre 1 — À retenir

Au cours de ce chapitre, nous avons vu :

- comment est constitué un système informatique ;
- les principaux composants matériels ;
- le rôle du système d'exploitation ;
- le rôle particulier du noyau ;
- la différence entre programme et processus ;
- les principales familles de systèmes d'exploitation.